

Topic 1: Inheritance, **Interfaces** and Design patterns

Part 2: Interfaces

Agenda

1. Introduction
2. What is an interface?
3. Interfaces to resolve multiple inheritance
4. Abstract class vs Interface
 - a. Similarity and difference
 - b. When to use interface or abstract class
5. Interface: Exercise
6. Summary
7. Questions and Comments

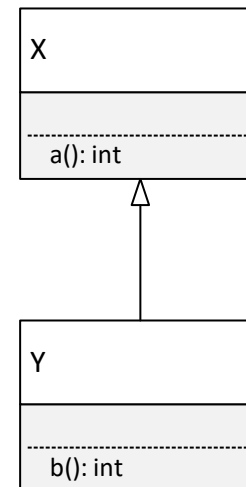
Announcement:

Exercise

Question 1

Class **X** declares a method **a()** (return type is int). Class **Y** inherits class **X**. In each of the following, state whether the statement is **true** or **false**. Explain your answer.

- (a) The method **a()** is abstract
(b) The method **a()** is NOT abstract
- (a) Class **X** extends **Y**
(b) Class **Y** extends **X**
- (a) Class **X** is an abstract class
(b) Class **Y** is an abstract class
- (a) **X** inherits and overrides method **a()**
(b) **Y** inherits and overrides method **a()**
- If **Y** does NOT implement **a()**, then:
(a) **Y** must be declared abstract
(b) **Y** must be declared concrete

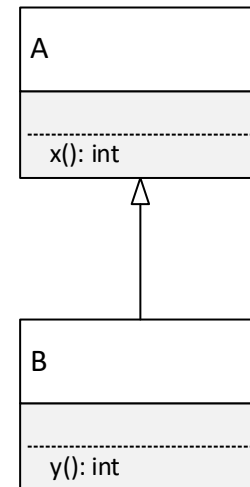


Exercise

Question 2

Class **A** declares a method **x()** (return type is int). Class **Y** inherits class **X** and declares a method **y()**. In each of the following, state whether the statement is **true** or **false**. Explain your answer.

1. If **B** implements both **x()** and **y()**, B is:
 - (a) a concrete class
 - (b) an abstract class
2. If **B** implements **x()** but not **y()**, B is:
 - (a) a concrete class
 - (b) an abstract class
3. If **B** implements ONLY **y()** but not **x()**, B is:
 - (a) a concrete class
 - (b) an abstract class



1. Introduction

Overview of previous lecture

1. Objects, encapsulation and classes
2. Relationships: Inheritance and Composition, and their benefits
3. Polymorphism, method overload and override
4. Concrete and Abstract classes
5. Multiple inheritance and problems
6. An **interface** is also an **important structure** in Java
7. Interfaces can be used to:
 - a. Separate “what” and “how”
 - b. Define a contract (between an interface and implementation class)
 - c. Resolve multiple inheritance
8. An interface is a special abstract class (similar but different from abstract class)

2. What is an interface?

- An interface is a special abstract class.
 - In general, an interface can contain:
 1. Constants
 2. Abstract methods (no implementations)
 3. Default methods (with implementations) *****
- It is declared with the keyword **interface** (not a class)
- All methods are by default **public** and **abstract**
- All variables are by default **public**, **static** and **final**
- Method implementation must be default****
- NOTE: some abstract classes can be converted to interfaces

2. What is an interface?

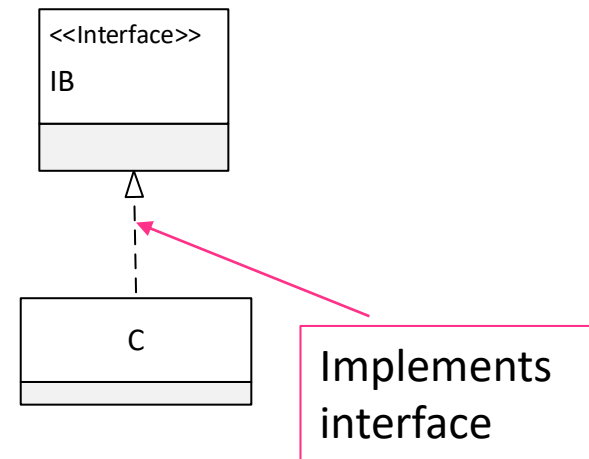
- A class **implements** an interface
- An interface is a **contract**:
 1. a class which **implements** an interface **implements all methods in the interface**
 2. The contract is enforced at **compile time**
- **Note**:
 1. Interfaces are commonly used to **declare related methods** and **public constants**
 2. An **interface** can **inherit another interface**
 3. A **class** (abstract or concrete) can **inherits** another class

2. What is an interface?

- The code snippets for the UML diagram

```
...  
public interface IB {  
    //write the code here  
    ...  
}  
  
...  
public class C implements IB {  
    //implement the abstract methods in IB  
    ...  
}  
  
...  
public static void main(String[] args) {  
    //IB i1=new IB();//cannot instantiate an interface  
    IB i2 =new C();//correct: C implements IB  
    ...  
}  
}
```

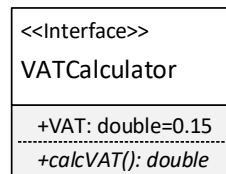
NOTE: declare with **interface** keyword



NOTE: write each code separately
in the package

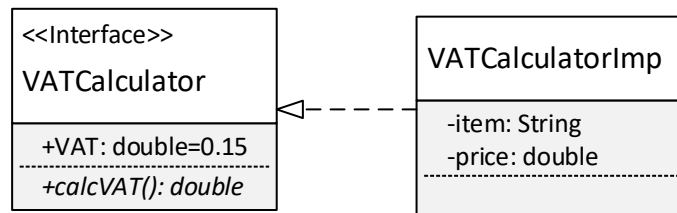
2. What is an interface?

- Example: The interface VATCalculator can calculate the VAT payable on an item



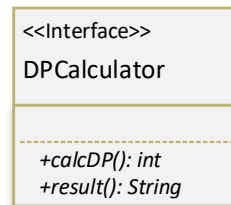
1. VAT is a constant, equal to 15%
2. calcVAT() returns the vat payable on an item, calculated as 15% of the price of an item

- Interface and implementation class



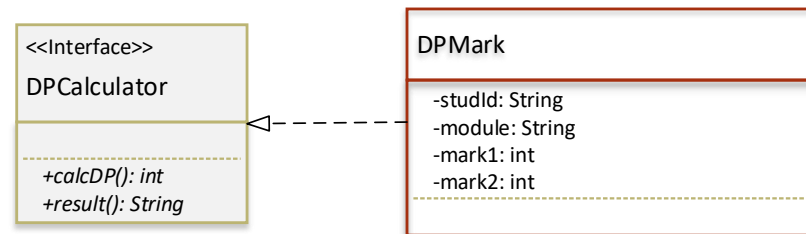
2. What is an interface?

- Example: The interface DPCalculator can calculate dp and determine the result



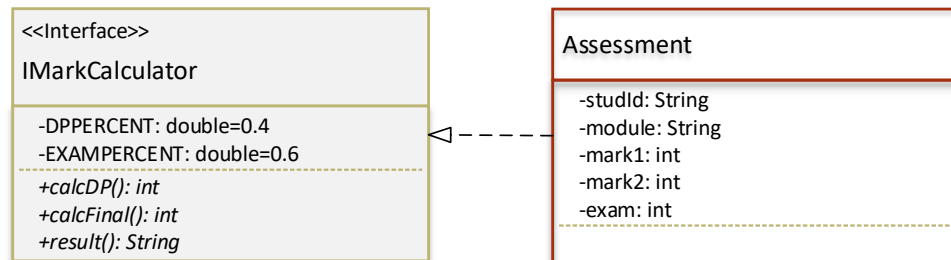
1. calcDp() returns the average mark
2. result() returns a “Admitted” if dp is ≥ 40 else “Not admitted”

- The interface and the implementation class



2. What is an interface?

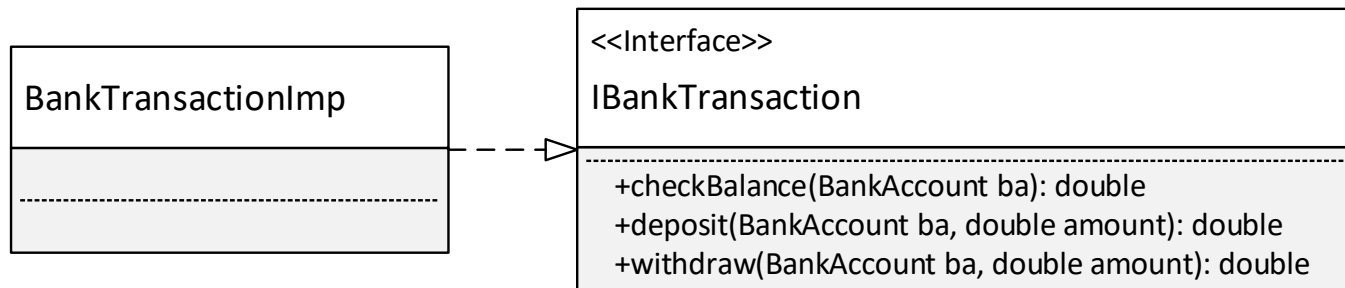
- Example: The interface IMarkCalculator can calculate dp and final mark



- The interface IMarkCalculator can calculate dp and final mark
 1. DPPERCENT is the percentage of DP (i.e., 40%)
 2. EXAMPERCENT is the percentage of exam mark (i.e., 60%)
 3. calcDp returns the dp (calculated as average of two marks)
 4. calcFinal() returns the final mark (as DPPERCENT of DP + EXAMPERCENT of exam)
 5. result() returns a “Pass” if final is ≥ 50 else “Fail”

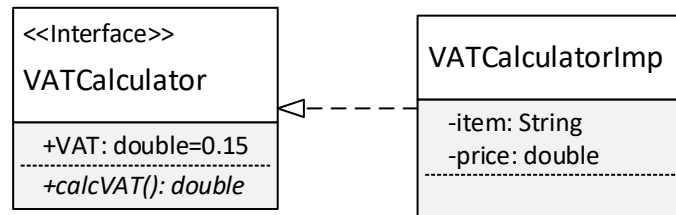
2. What is an interface?

- Example: Interface showing “what” are the methods in a bank account



2. What is an interface?

- Exercise: Below is an interface and implementation class.

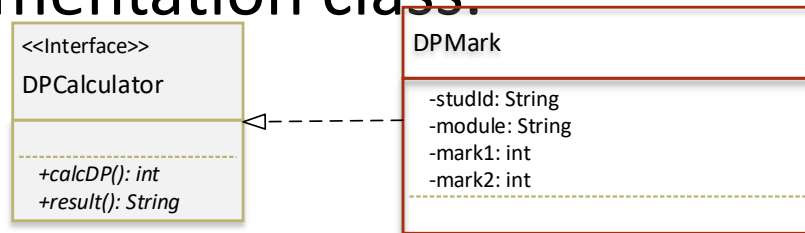


1. Write the code for the interface (VAT is 15%)
2. Write the implementation class. `calcVAT()` returns the VAT payable on an item (calculated as 15% of the price of an item)
3. Write a Tester to test the interface and implementation class. A typical output is shown below:

Item=Meat Price=100.0 VAT (@15% of price)=15.0

2. What is an interface?

- Exercise: Below is an interface and implementation class.



- Write the code for the interface
- Write the implementation class. `calcDp()` returns the average mark. `result()` returns a “Admitted” if dp is ≥ 40 else “Not admitted”
- Write a Tester to test the interface and implementation class. A typical output is shown below:

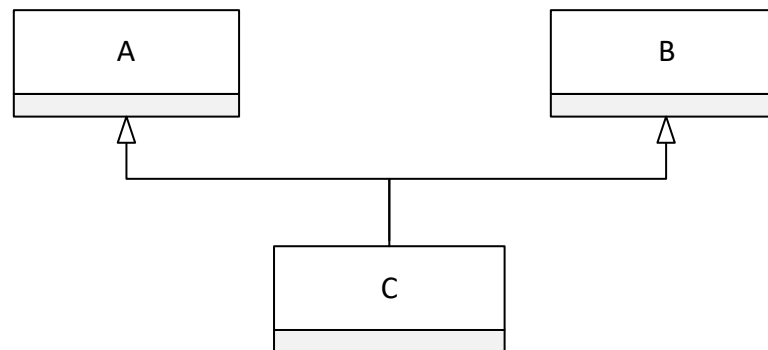
StudId=20202020 Module=IPRT

Mark1=20 Mark2=30

DP=25 Result=Not Allowed

3. Interface to resolve multiple inheritance

- Multiple inheritance allows a subclass to inherit the members of two or more super classes
- Problem with multiple inheritance: conflicts can arise from references to members of the super classes.
 - E.g., a subclass **C** inherits **toString()** from two super classes **A** and **B**.
 - **Problem:** Which **toString()** to **call at run time**?
 - The programmer needs to manage calls to members of super classes to avoid errors

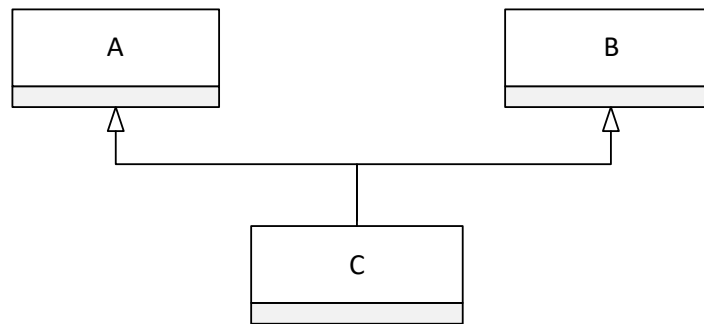


3. Interfaces to resolve multiple inheritance

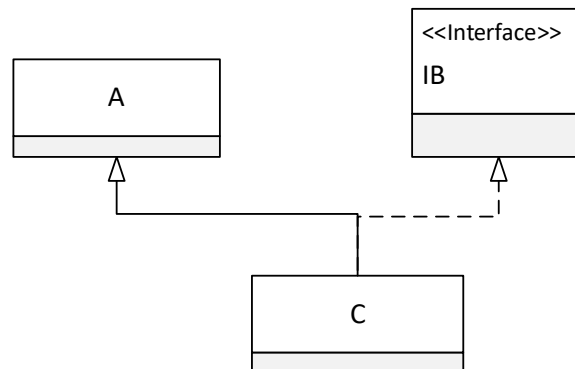
- Java does not support multiple inheritance (unlike other programming languages, e.g., Python)
- A Java class can **inherit** from ONLY one superclass
- One way around multiple inheritance is to use interfaces
 - Keep only one super class; **convert other super classes to interfaces**
 - This allows a subclass to **inherit** only one superclass but **implement** many interfaces

3. Interfaces to resolve multiple inheritance

- Example: The diagram below shows class C inherits classes A and B

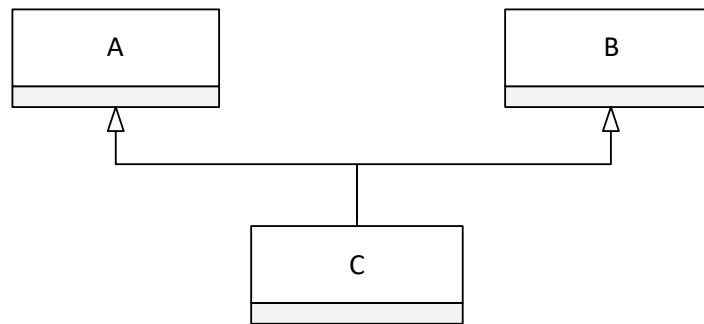


1. Explain why you cannot convert the diagram to a Java program
2. If B can be converted to an interface, draw the UML diagram

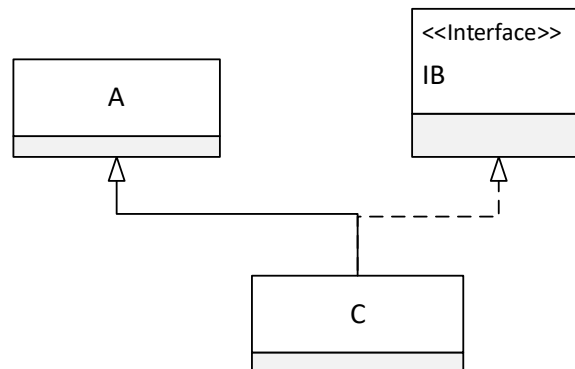


3. Interfaces to resolve multiple inheritance

- Example: The diagram below shows class C inherits classes A and B



1. Explain why you cannot convert the diagram to a Java program
2. If B can be converted to an interface, draw the UML diagram



3. Interfaces to resolve multiple inheritance

- The code snippets for the UML diagram

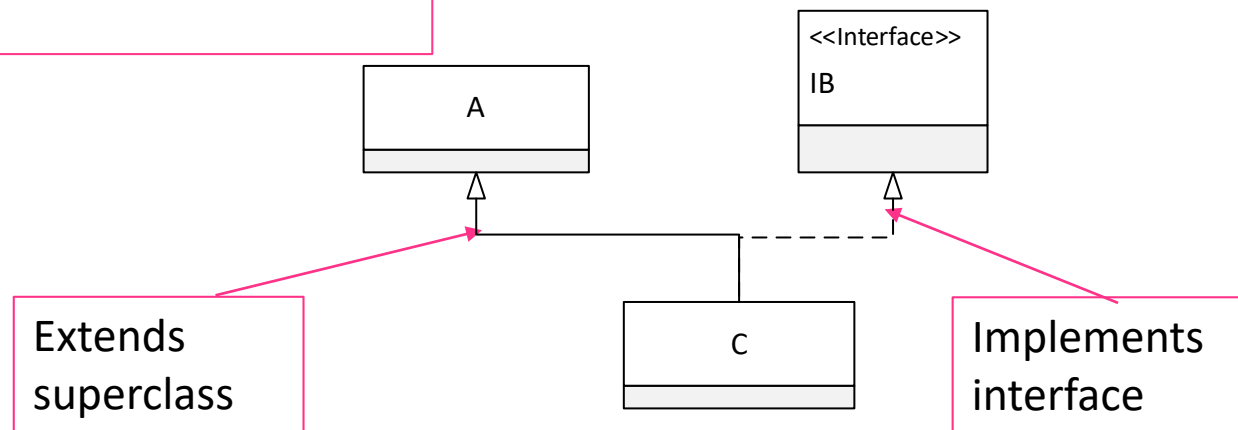
```
public interface IB {  
    //...  
}
```

```
public class A {  
    //...  
}
```

```
public class C extends A implements IB{  
    //...  
}
```

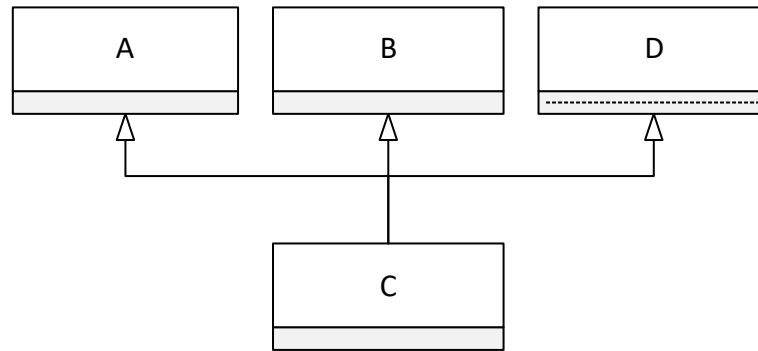
```
public class TestC {  
    public static void main(String[] args) {  
        //...  
    }  
}
```

NOTE: write each code and save in a separate file



3. Interfaces to resolve multiple inheritance

- The UML diagrams below contains multiple inheritance.



- To remove multiple inheritance, B and D are converted to interfaces IB and ID. If C inherits A and implements IB and ID,
 1. Write the class header of A
 2. Write the interface header of IB
 3. Write the interface header of ID
 4. Write the class header of C
- **Remember:** not all classes can be converted to interfaces

3. Interfaces to resolve multiple inheritance

- Code segment to declare the classes and interface

Code and save
each separately

```
public class A {  
    //  
}  
public interface IB {  
    //  
}  
public interface ID {  
    //  
}  
public class C extends A implements IB, ID {  
    //  
}  
public class TestABCD {  
    public static void main(String[] args) {  
        //  
    }  
}
```

4. Abstract class vs Interface

1. A class implements interface; an interface cannot implement an interface
2. An interface inherits another interface; a class (concrete or abstract) inherits another class
3. When to use interface or abstract class?
 - a. Interface to resolve multiple inheritance
 - b. Interface to separate “what” from “how”
 - c. Otherwise use the guide below on what each can contain:

Abstract class
instant variables static variables constants (final) static constants

methods implementation/definition static methods abstract methods abstract static methods

<<interface>>
static constants (final)

private methods*** static methods*** default methods (method definitions)*** abstract methods

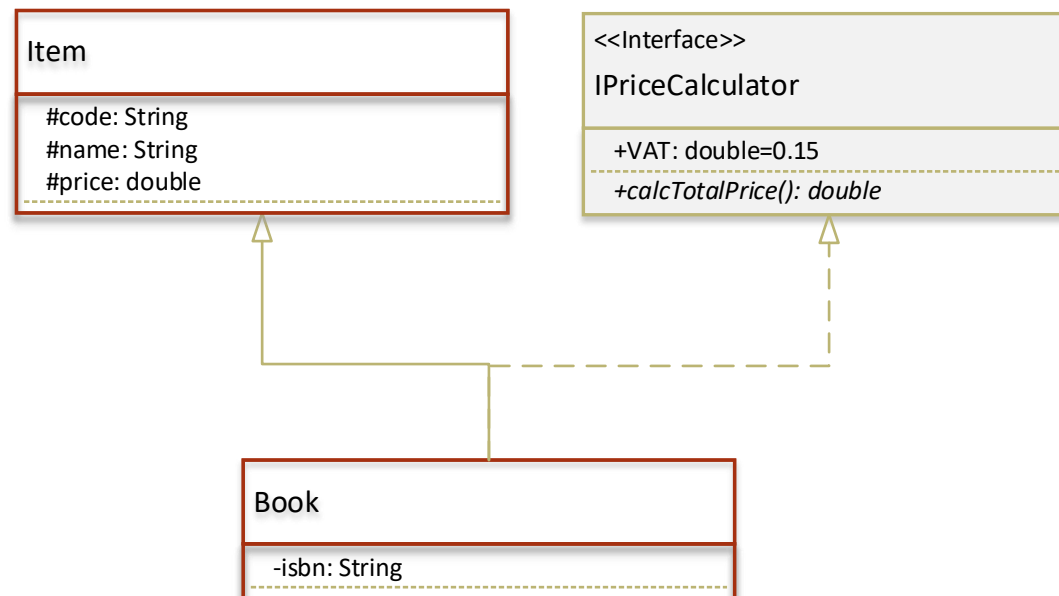
4. Abstract class vs Interface

Summary

	Abstract class	Interface
Instance variable	Yes	No
Constants	Yes	Yes (static final)
Constructor	Yes	No
Method implementation	Yes	**Yes, declare as default**
Abstract method	Yes	Yes
Static methods	Yes	Yes
Inherited	Yes (by a class)	Yes (by an interface)
Implemented	No	Yes (by class)
Instantiation	No	No

5. Interface (see Exercise 4- Interfaces)

1. Convert the UML diagram to code.
 2. Write a program to test Item, PriceCalculator and Book
 1. VAT is a constant (15%)
 2. calcTotalPrice() returns the total price, i.e., price and VAT payable, calculated as 15% of the price (i.e., total price= price+ price *0.15)
- Question: **should the class Item be abstract or not? Why?**



5. Interface: Exercise

- Below is the structure of programs

Item.java

Item

- code
- name
- price
- Item()
- Item(String, String, double)
- getCode() : String
- getName() : String
- getPrice() : double
- setCode(String) : void
- setName(String) : void
- setPrice(double) : void
- toString() : String

IPriceCalculator.java

IPriceCalculator

VAT

calcTotalPrice() : double

Tester.java

Tester

main(String[]) : void

Book.java

Book

- isbn
- Book()
- Book(String, String, double, String)
- calcTotalPrice() : double
- toString() : String

5. Interface: exercise

- Typical output is shown below:

Item code	BF0001
Item name	My book
Item price	120.0
Isbn	1-4239-0135-5
Total Price	138.0

6. Summary

1. Multiple inheritance and problems
2. Interface as solution
3. The main features of interface
4. How to code and use an interface
5. Interface vs abstract class
 1. The similarity and differences
 2. When to use interface or abstract class
6. Examples
7. Before you go:
 - What is POCO (and POJO)
 - Most of our programming is based on Pure Old Class Object (Pure Old Java Object)

7. Questions and comments



Next.....

- Part 3: Design patterns